

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#01)

INTRODUCTION TO VERILOG

Student Name: M SHAFAY UL HASSAN Roll No: 24I-6552 Section: CE-A

Date performed: 18-08, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:
Implemented the 7 basic gates with the Gate Level approach. Gate Level approach is that you have to implement the gates with the gates keyword like and,or,not etc.
Code:
<pre>module AND_gate(a,b,c); //AND Gate input a; input b; output c; and chk(c,a,b); endmodule module AND_test_bench();// the input vars would be of reg type and the output vars would be of wire type reg x; reg y; wire z; AND_gate uu(.a(x),.b(y),.c(z)); initial begin x=0;y=0; #50 x=1;y=0; #50 x=0;y=1; #50 x=1;y=1; end endmodule // OR Gate module OR_gate(a,b,c); //or Gate input a; input b; output c; or chk(c,a,b); endmodule module OR_test_bench();// the input vars would be of reg type and the output vars would be of wire type reg x; reg y; wire z; OR_gate uu(.a(x),.b(y),.c(z)); initial begin x=0;y=0; #50 x=1;y=0; #50 x=0;y=1; #50 x=1;y=1; end endmodule //not gate module NOT_gate(a,c); //AND Gate input a;</pre>

```
output c;
not chk(c,a);

endmodule

module NOT_test_bench();// the input vars would be of reg type and the output vars
would be of wire type
reg x;
wire z;
NOT_gate uu(.a(x),.c(z));
initial
begin
x=0;
#50 x=1;
end
endmodule

//NoR Gate
module NOR_gate(a,b,c);
input a;
input b;
output c;
nor chk(c,a,b);

endmodule

module NOR_test_bench();// the input vars would be of reg type and the output vars
would be of wire type
reg x;
reg y;
wire z;
NOR_gate uu(.a(x),.b(y),.c(z));
initial
begin
x=0;y=0;
#50 x=1;y=0;
#50 x=0;y=1;
#50 x=1;y=1;
end
endmodule

//XOR Gate
module XOR_gate(a,b,c);
input a;
input b;
output c;
xor chk(c,a,b);

endmodule

module XOR_test_bench();// the input vars would be of reg type and the output vars
would be of wire type
reg x;
reg y;
wire z;
XOR_gate uu(.a(x),.b(y),.c(z));
initial
begin
x=0;y=0;
#50 x=1;y=0;
#50 x=0;y=1;
#50 x=1;y=1;
end
endmodule
```

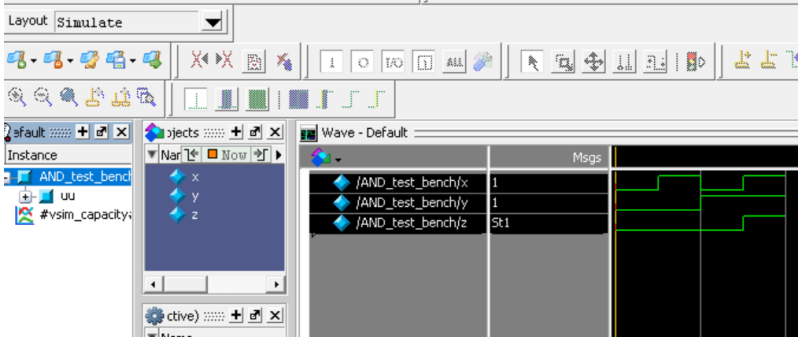
```
//NAND Gate
module NAND_gate(a,b,c);
input a;
input b;
output c;
nand chk(c,a,b);

endmodule
module NAND_test_bench();// the input vars would be of reg type and the output vars
would be of wire type
reg x;
reg y;
wire z;
NAND_gate uu(.a(x),.b(y),.c(z));
initial
begin
x=0;y=0;
#50 x=1;y=0;
#50 x=0;y=1;
#50 x=1;y=1;
end
endmodule
// XNOR Gate
module XNOR_gate(a,b,c);
input a;
input b;
output c;
xnor chk(c,a,b);

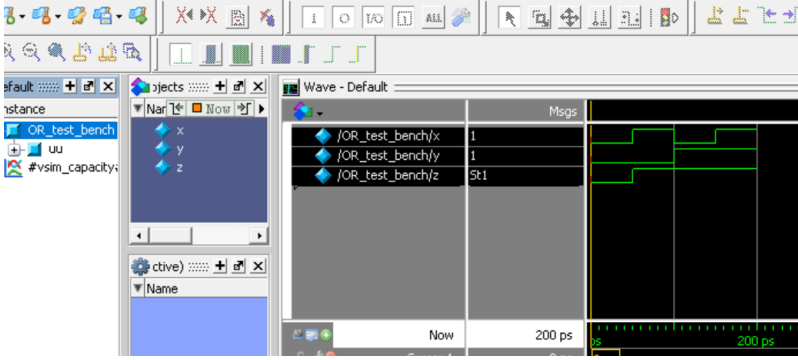
endmodule
module XNOR_test_bench();// the input vars would be of reg type and the output vars
would be of wire type
reg x;
reg y;
wire z;
XNOR_gate uu(.a(x),.b(y),.c(z));
initial
begin
x=0;y=0;
#50 x=1;y=0;
#50 x=0;y=1;
#50 x=1;y=1;
end
endmodule
```

Output:

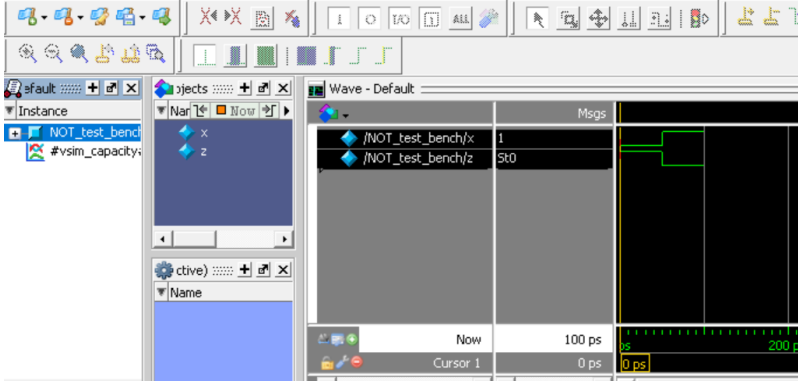
AND Gate:



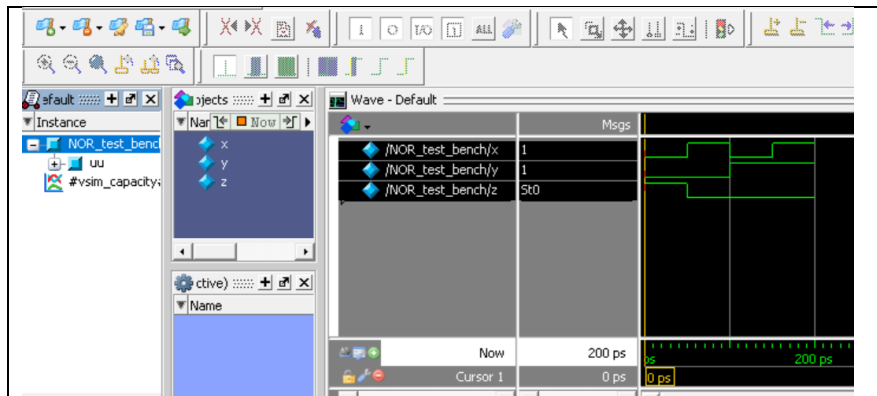
OR Gate:



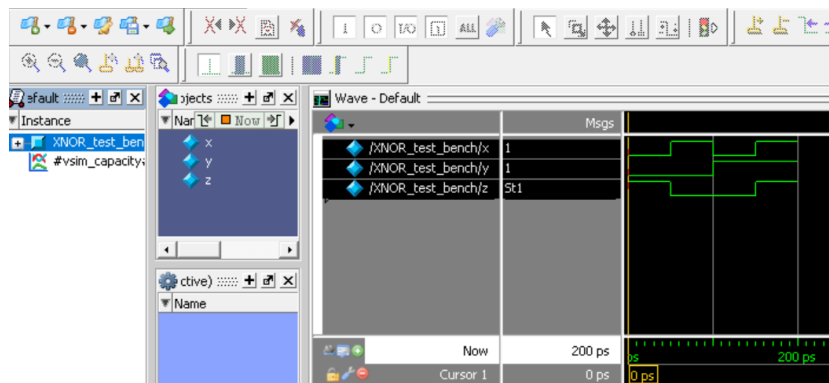
NOT Gate:



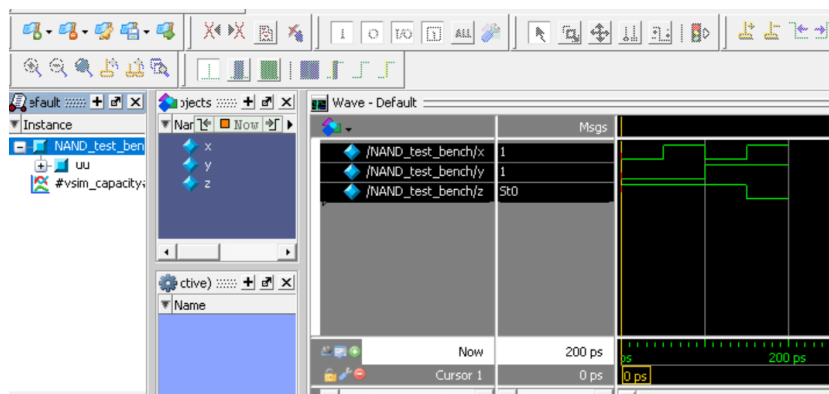
NOR Gate:



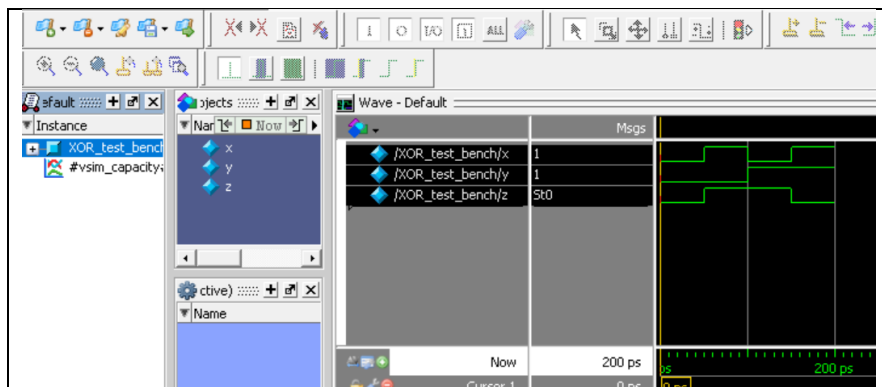
XNOR Gate:



NAND Gate:



XOR Gate:



Task 2:

Description:

Implemented the 7 basic gates with the Data Flow Level Approach Data Flow Level approach is that you have to implement the gates with the assign keyword like assign c=a&b.

Code:

```
module data_OR_gate(a,b,c); //or Gate
input a;
input b;
output c;
assign c=a|b;

endmodule

module data_OR_test_bench(); // the input vars would be of reg type and the output vars
would be of wire type
reg x;
reg y;
wire z;
data_OR_gate uu(.a(x),.b(y),.c(z));
initial
begin
x=0;y=0;
#50 x=1;y=0;
#50 x=0;y=1;
#50 x=1;y=1;
end
endmodule

//and Gate
module data_AND_gate(a,b,c);
input a;
input b;
output c;
assign c=a&b;

endmodule

module data_AND_test_bench();// the input vars would be of reg type and the output
vars would be of wire type
```

```
reg x;
reg y;
wire z;
data_AND_gate uu(.a(x),.b(y),.c(z));
initial
begin
x=0;y=0;
#50 x=1;y=0;
#50 x=0;y=1;
#50 x=1;y=1;
end
endmodule
//not Gate
module data_NOT_gate(a,c);
input a;
output c;
assign c=~a;

endmodule
module data_NOT_test_bench();// the input vars would be of reg type and the output
vars would be of wire type
reg x;
wire z;
data_NOT_gate uu(.a(x),.c(z));
initial
begin
x=0;
#50 x=1;
#50;
end
endmodule
//NOR Gate
module data_NOR_gate(a,b,c);
input a;
input b;
output c;
assign c=~(a|b);

endmodule
module data_NOR_test_bench();// the input vars would be of reg type and the output
vars would be of wire type
reg x;
reg y;
wire z;
data_NOR_gate uu(.a(x),.b(y),.c(z));
initial
begin
x=0;y=0;
#50 x=1;y=0;
#50 x=0;y=1;
#50 x=1;y=1;
end
endmodule
//XOR Gate
module data_XOR_gate(a,b,c);
input a;
input b;
output c;
assign c=a^b;
```



```
endmodule
module data_XOR_test_bench();// the input vars would be of reg type and the output
vars would be of wire type
reg x;
reg y;
wire z;
data_XOR_gate uu(.a(x),.b(y),.c(z));
initial
begin
x=0;y=0;
#50 x=1;y=0;
#50 x=0;y=1;
#50 x=1;y=1;
end
endmodule
//Nand Gate
module data_NAND_gate(a,b,c);
input a;
input b;
output c;
assign c=~(a&b);

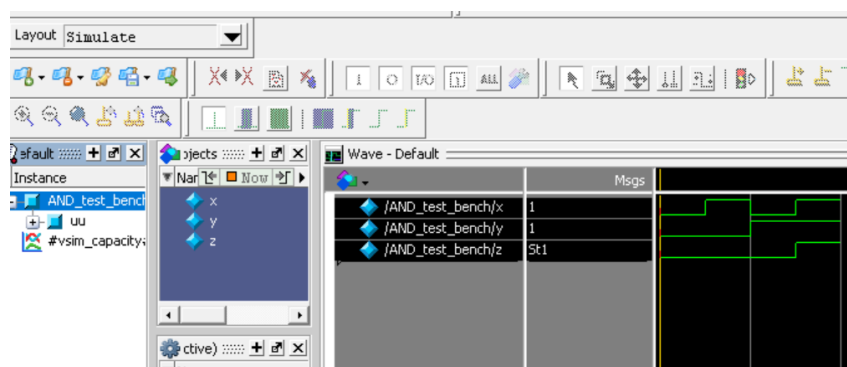
endmodule
module data_NAND_test_bench();// the input vars would be of reg type and the output
vars would be of wire type
reg x;
reg y;
wire z;
data_NAND_gate uu(.a(x),.b(y),.c(z));
initial
begin
x=0;y=0;
#50 x=1;y=0;
#50 x=0;y=1;
#50 x=1;y=1;
end
endmodule
//XNOR Gate
module data_XNOR_gate(a,b,c);
input a;
input b;
output c;
assign c=~(a^b);

endmodule
module data_XNOR_test_bench();// the input vars would be of reg type and the output
vars would be of wire type
reg x;
reg y;
wire z;
data_XNOR_gate uu(.a(x),.b(y),.c(z));
initial
begin
x=0;y=0;
#50 x=1;y=0;
#50 x=0;y=1;
#50 x=1;y=1;
end
end
```

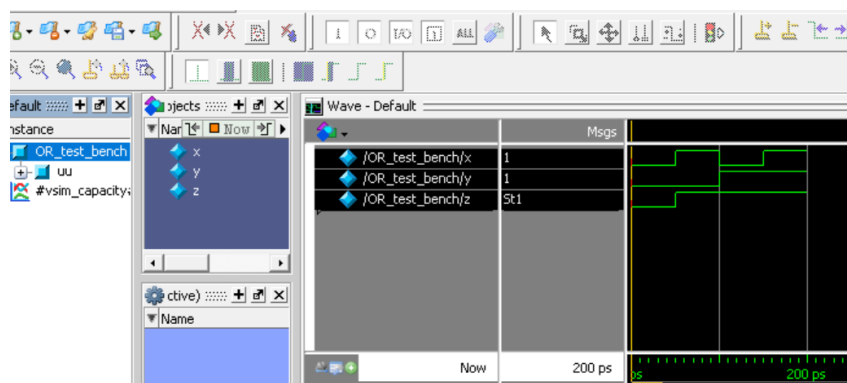
endmodule

Output:

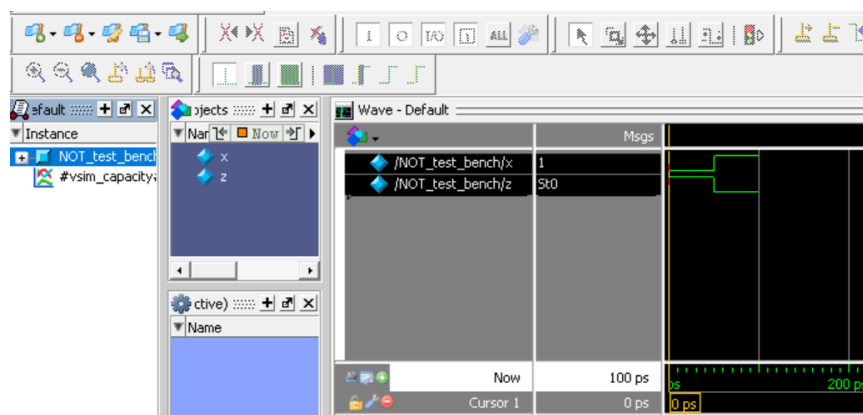
AND Gate:



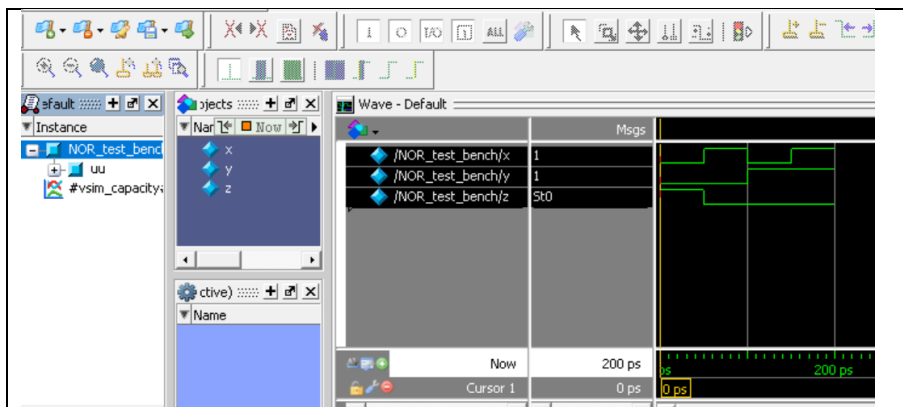
OR Gate:



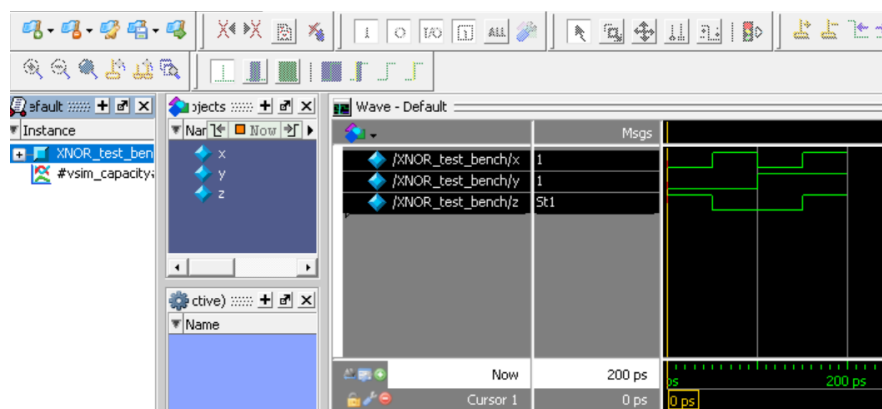
NOT Gate:



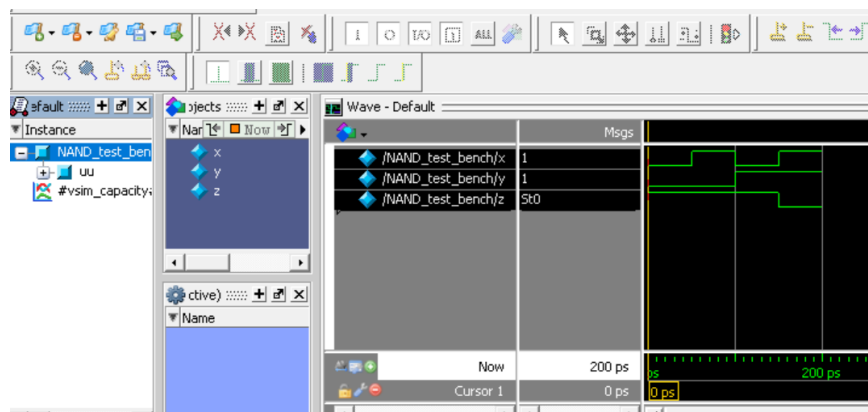
NOR Gate:



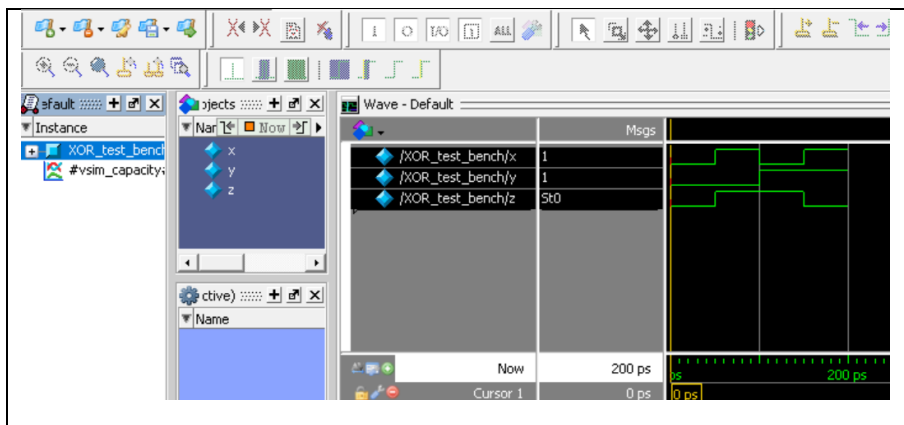
XNOR Gate:



NAND Gate:



XOR Gate:

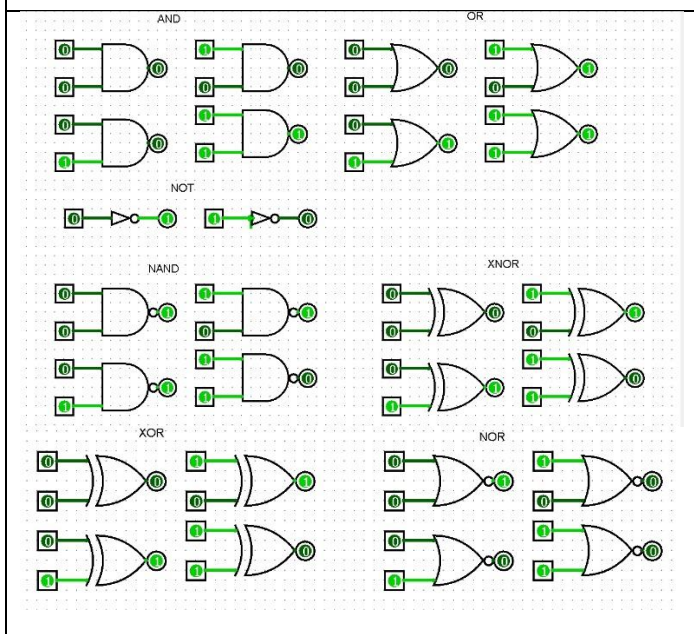


Task 3:

Description:

Implemented the gate level and data flow level of Verilog codes of the first task in logeism to verify the results in both the logism and modelsim . This technique is very useful to verify your results.

Output:



Task 4:

Description:

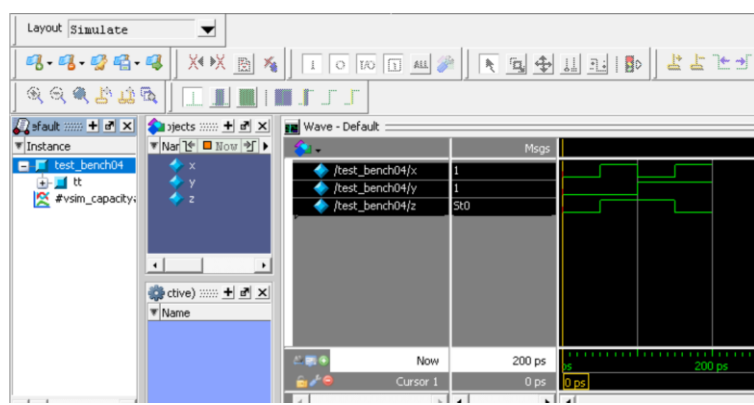
Implemented the 7 basic gates with the Data Flow Level Approach Data Flow Level approach is that you have to

implement the gates with the assign keyword like assign
 $c=a \& b$. Solved the complex circuit by using wires and output
and input keywords .

Code:

```
module comp_big(a,b,out);  
input a;  
input b;  
output out;  
wire t1;  
wire t2;  
wire ta;  
wire tb;  
not temp1(t1,a);  
and temp2(ta,t1,b);  
not temp3(t2,b);  
and temp4(tb,a,t2);  
or tt(out,ta,tb);  
endmodule  
module test_bench04();  
reg x;  
reg y;  
wire z;  
comp_big tt(.a(x),.b(y),.out(z));  
initial  
begin  
x=0;y=0;  
#50 x=1;y=0;  
#50 x=0;y=1;  
#50 x=1;y=1;  
#50;  
end  
endmodule
```

Output:

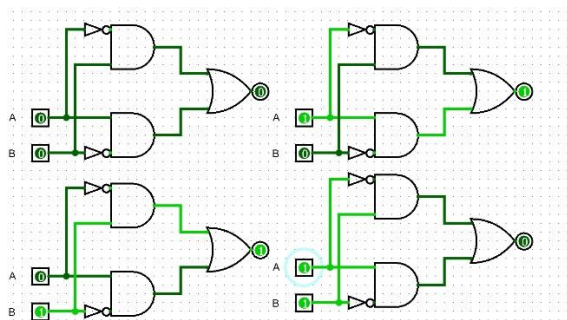


Task 5:

Description:

Implemented the gate level and data flow level of Verilog codes of the first task in logeism to verify the results in both the logism and modelsim . This technique is very useful to verify your results .

Output:



Task 6:

Description:

Implemented the gate level approach to solve the one's compliment problem by taking the 4 inputs and 4 outputs and check the result using the waveforms .Also used wires in the test bench for the simulation results.

Code:

```
module comp_big(a,b,out);
input a;
input b;
output out;
wire t1;
wire t2;
wire ta;
wire tb;
not temp1(t1,a);
and temp2(ta,t1,b);
not temp3(t2,b);
and temp4(tb,a,t2);
or tt(out,ta,tb);
endmodule
module test_bench04();
reg x;
reg y;
wire z;
comp_big tt(.a(x),.b(y),.out(z));
initial
begin
x=0;y=0;
```

```
#50 x=1;y=0;  
#50 x=0;y=1;  
#50 x=1;y=1;  
#50;  
end  
endmodule
```

Output:

